LOOPGYM: TRAINING FOR COMBINED INFERENCE IN TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Loop-invariant generation is compositional: one response may contain a useful bound, another a relational equality, and neither need prove the target alone. This makes pass@1 a poor training and evaluation objective for an inference procedure that can combine partial answers. We study this problem under *target-hidden loop verification*, where the model sees the precondition, guard, and body but not the assertion or postcondition Q. Our inference procedure samples multiple responses, splits conjunctive invariants into clauses, unions them, iteratively removes syntax failures, and uses verifier-checked Houdini pruning to retain an inductive subset before Q is restored for the final proof. We align both stages of training with this rollout–combine–filter procedure. First, synthetic supervised targets are formed by applying it to multiple target-hidden responses and serializing the combined verified set, thereby compressing a multi-response search result into one response. Second, reinforcement learning rewards two properties needed by the same inference procedure: the behavioral strength of each response and its complementary contribution within a rollout group. The reward uses reachable executions and conservatively filtered synthetic negative examples, never Q. The resulting objective is not to maximize standalone proof rate, but to train responses that combine into a stronger verified set under a fixed inference budget. On 832 C programs, five-rollout inference verifies 74.76% of tasks, compared with 61.78% for the strongest external baseline. Reinforcement learning raises combine@10 from 52.40% to 56.97% even as pass@1 decreases, and reduces the rollouts needed to reach 95% of the measured maximum from 30 to 10. Thus ten trained-model responses outperform the combination of one hundred base-model responses.

1 INTRODUCTION

Verifying any program with a loop requires finding an adequate invariant: no proof can otherwise cover arbitrarily many iterations, and writing invariants by hand remains a central bottleneck to verifying real software. Loop invariants summarize an unbounded computation in a finite assertion. They must be inductive while retaining enough program knowledge to establish the desired property at loop exit. A weak predicate such as $true$ can satisfy inductiveness but prove nothing useful. We formalize soundness, logical strength, and target sufficiency in Section 2.

Invariant generation is compositional: one response may provide a bound and another a relational equality, so their union can prove a target that neither proves alone. Pass@1 discards this partial progress. We instead sample multiple responses and retain a verified subset of their combined clauses with our rollout–combine–filter framework (Section 4.2).

This procedure needs both *strength* and *exploration*: each response should contain useful clauses, while different responses should cover different relations. Optimizing standalone success instead encourages repetition and ignores partial answers. Our objective is the filtered combination at a fixed rollout budget, ideally compressing large-budget exploration into one practical deployment run.

We study this alignment problem under *target-hidden loop verification*, where the target is withheld until after the invariant set has been fixed (Section 2). Existing methods can use the target, failed obligations, or counterexamples to guide search (Gulwani et al., 2008; Gupta & Rybalchenko, 2009; Dillig et al., 2013; Alur et al., 2013; Garg et al., 2014; 2016; Si et al., 2020; Kamath et al., 2023; Cao

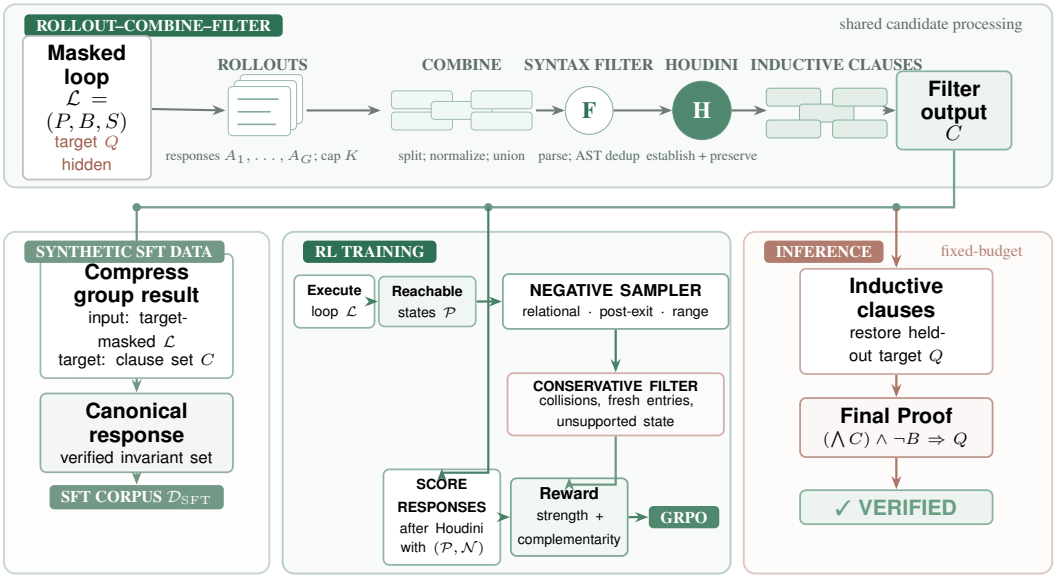

Figure 1: **Training is organized around rollout–combine–filter inference.** Multiple target-hidden responses are split into clauses, capped, normalized, and combined. The filter iteratively removes syntax failures, optionally performs AST deduplication, then uses Houdini to iteratively remove clauses that fail establishment or preservation. SFT compresses the result into one supervised response; RL rewards strong and complementary responses. Deployment restores Q only after filtering.

et al., 2025). Hiding Q prevents copying an inductive postcondition. Final success is unchanged: the verifier must prove soundness and sufficiency for the restored target.

LoopGym aligns both training stages with this structure. For SFT, it applies the same framework, then serializes the verified set as one target for the masked program. This distills multi-response search into a stronger response for later combined inference.

Within loop-invariant generation specifically, prior systems search, filter counterexamples, or rank candidates, but none trains a generation policy with reinforcement learning (Section 3). A usable reward is itself hard to define once Q is hidden: binary inductiveness cannot separate a useful invariant from a weak one such as $true$, and querying Q during training would defeat the target-hidden setting. For RL, executions provide reachable states and conservative negative examples without querying Q. After Houdini, negative coverage rewards response strength; a closed-form Shapley allocation divides the group's combined negative coverage among the responses that provide it. Together they approximate the strength and complementarity needed after combination.

Our experiments deliberately separate standalone success from combined inference. On 832 integer C programs, five target-hidden responses with union and Houdini verify 622 programs (74.76%), compared with 514 (61.78%) for the strongest external baseline. On Qwen3-8B, reinforcement learning decreases direct pass@1 from 9.30% to 4.88% but increases combine@10 from 52.40% to 56.97%. Ten responses from the trained policy exceed combine@100 of the base model, reducing the rollouts needed to reach 95% of the measured maximum from 30 to 10. Training thus compresses broad base-model exploration into a much smaller deployment budget rather than optimizing one standalone target-proving answer.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification as a compositional generation problem and evaluate the filtered combination of multiple responses, rather than treating pass@1 as the deployment objective.

- We align training with rollout–combine–filter inference. Synthetic SFT targets distill filtered combinations into individual responses, while an execution-based RL reward balances per-response strength with complementary contributions across a rollout group without observing Q.
- We show that this alignment improves both combined verification and rollout efficiency, and audit the synthetic negative examples that support the reward.

2 PRELIMINARIES

2.1 LOOP VERIFICATION

A verification task consists of a loop $\mathcal{L} = (P, B, S)$ and a postcondition Q, where P is the precondition, B is the loop guard, and S is the loop body. Let $\mathrm{Reach}(\mathcal{L})$ denote the states reachable at the loop head, including the final guard test at loop exit.

An invariant I is *sound* (equivalently, inductive) when it is established by P and preserved by one iteration:

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\}.$$

These obligations ensure that I holds on every state in $\mathrm{Reach}(\mathcal{L})$ (Floyd, 1967; Hoare, 1969).

We say that I_1 is *stronger* than I_2, written $I_1 \succeq I_2$, when

$$\models I_1 \Rightarrow I_2.$$

An invariant $I^\star$ is *exact* when

$$[\![I^\star]\!] = \mathrm{Reach}(\mathcal{L}).$$

Because every sound invariant contains all reachable states, an exact invariant is stronger than every sound invariant of the same loop:

$$I^\star \text{ exact},\ J \text{ sound} \quad \Longrightarrow \quad \models I^\star \Rightarrow J.$$

Exactness is a semantic ideal; the reachable states need not have a finite representation in the annotation fragment used by the generator.

Finally, a sound invariant I is *Q-sufficient* when it proves the postcondition at loop exit:

$$I \wedge \neg B \Rightarrow Q.$$

Thus exactness describes the loop independently of a target, whereas Q-sufficiency is relative to one postcondition. The final verifier must prove both soundness obligations and this exit obligation.

2.2 TARGET-HIDDEN VERIFICATION

In *target-hidden loop verification*, the generator and all intermediate feedback see $\mathcal{L} = (P, B, S)$ but not Q. The postcondition is restored only after the generated invariant has been fixed. Preconditions and executable loop semantics remain visible, while assertions, postconditions, assertion calls, and conventional error-label names are masked. The untouched program is used only for final verification.

2.3 SCOPE AND MOTIVATING EXAMPLE

We focus on numerical loops for two reasons. First, their invariants are predominantly arithmetic equalities and inequalities, including polynomial relations. These predicates are comparatively amenable to automatic checking, which provides a scalable and objective verification signal. Second, numerical loops factor out the representational machinery of arrays, heaps, and recursive data structures, whose verification often requires auxiliary functions or inductive predicates. This gives a controlled setting in which success depends more directly on discovering algebraic relations and reasoning about loop dynamics.

Concretely, we study one braced `while` loop over scalar C integers, including nonlinear updates; arrays, heaps, and recursive structures are out of scope. This restriction does not make invariant discovery trivial: even a small loop can hide a nonlinear relation across coupled recurrences.

Consider the following abridged program:

```
int x = 1, y = 1, c = 1;
while (c < k) { c++; x = x*z + 1; y = y*z; }
/*@ assert 1+x*z-x-z*y == 0; */
```

The visible assertion can be copied verbatim as an invariant; when hidden, its nonlinear relation must instead be recovered from the coupled updates.

3 Related Work

Programmatic invariant synthesis. Classical methods infer invariants with abstract interpretation, affine equalities, templates, interpolation, recurrences, or constraint solving (Karr, 1976; Cousot & Cousot, 1977; Cousot & Halbwachs, 1978; Sankaranarayanan et al., 2004; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017). These methods provide strong guarantees for selected languages, especially affine transition systems, but nonlinear templates trade generality for increasingly difficult algebraic search. Dynamic approaches instead infer likely relations from execution traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). LoopGym also uses executions, but only to construct a training signal; final soundness is established independently by the verifier. Target-directed software model checkers constitute a related but distinct setting. For example, Ultimate Automizer (UAuto) uses trace abstraction and interpolation to verify a supplied safety property (Heizmann et al., 2013). Although its correctness witnesses may contain inductive assertions, they are synthesized with the verification target visible. UAuto therefore does not address our target-hidden setting, in which the assertion is withheld throughout invariant generation, and we do not include it in the experimental comparison.

Neural and LLM-based invariant generation. Learned methods broaden the candidate language by using neural generators, counterexamples, or verifier feedback (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020). Loopy generates candidate clauses, filters them with Houdini, and repairs failures using verifier feedback (Kamath et al., 2023). Clause2Inv combines generated clauses under counterexample guidance, and iRank learns to rank LLM-generated invariants before expensive verifier calls (Cao et al., 2025; Chakraborty et al., 2023). These systems solve a supplied target and commonly evaluate whether one response proves it. We hide the target during generation and training, and train for the filtered combination of compatible clauses from multiple responses.

Specification generation. LLM-based tools also synthesize broader program specifications. AutoSpec repeatedly proposes and verifies annotations; SpecGen combines LLM proposals with mutation in Java/JML; and SESpec uses symbolic execution to generate goal-independent specifications for C (Wen et al., 2024; Ma et al., 2025; Yang et al., 2026). We study only loop invariants and ask whether they prove a held-out target, rather than evaluating a general behavioral specification.

Reinforcement learning from formal feedback. Verifier-guided reinforcement learning has been used for program synthesis and formal reasoning. Re:Form uses scalable curation and verifier-guided RL for Dafny, while SpecRL generates impossible input–output pairs and rewards specifications that reject them (Yan et al., 2025; Huang et al., 2026). Validity alone does not distinguish a useful specification from a weak one. We specialize this idea to loops: synthetic negative states measure strength, while the verifier proves inductiveness. Verifiable-reward RL can also improve small-budget accuracy while narrowing extensive-sampling coverage (Yue et al., 2025). Because deployment unions samples, our reward adds group-relative credit for less common behavior that survives Houdini.

4 Method

4.1 Overview

Using the notation of Section 2, the model receives only the target-masked loop $\mathcal{L}$. We parse each response, split every invariant at its top-level conjunctions, normalize the resulting clauses, and obtain

Input: masked loop $\mathcal{L}$, responses $A_{1:G}$, cap K, AST-dedup flag d
Output: filtered clause set C

$C \leftarrow \bigcup_{i=1}^{G} \mathrm{Cap}_K(\mathrm{Normalize}(\mathrm{Split}_\wedge(A_i)))$
repeat
  $E \leftarrow \mathrm{ParserErrors}(\mathrm{Annotate}(\mathcal{L}, C))$
  $C \leftarrow C \setminus E$
until $E = \varnothing$ or $C = \varnothing$
if $C = \varnothing$ **then return** C
if d **then** $C \leftarrow \mathrm{ASTDedup}(C)$
repeat
  $V \leftarrow \mathrm{WP}_{\mathrm{est,pres}}(\mathcal{L}, C)$
  $D \leftarrow \{c \in C : \neg V_{\mathrm{est}}(c) \vee \neg V_{\mathrm{pres}}(c)\}$
  $C \leftarrow C \setminus D$
until $D = \varnothing$ or $C = \varnothing$
return C

Algorithm 1: Rollout–combine–filter

an ordered list $A = (a_1, \ldots, a_m)$. We cap each response at K clauses,

$$\bar{A} = (a_1, \ldots, a_{\min(m,K)}), \qquad o(A) = \max(0, m - K),$$

to prevent a response from gaining coverage by producing an unbounded list.

Our *rollout–combine–filter* framework is shared by data construction and deployment. It samples responses, combines their clauses, iteratively removes syntax failures, optionally performs AST deduplication, and then applies Houdini. SFT turns the filtered combination into one target, RL improves the responses that populate a combination, and inference applies the same framework before restoring Q.

4.2 Rollout–Combine–Filter

Rollout. The model emits ACSL (Baudin et al., 2021) annotations of the form `loop invariant <expr>;`. We split each `<expr>` at top-level logical conjunctions, so a conjunction contributes separate clauses that can survive or fail independently. Splitting precedes capping and normalization.

Combine. Given G responses, we union their clause lists:

$$X_0 = \bigcup_{i=1}^{G} \bar{A}_i.$$

This combination preserves partial progress across responses but can contain syntactically invalid or non-inductive clauses.

Filter. Algorithm 1 gives the full framework. The filter first iteratively removes clauses identified by the verifier's parser or type checker. It may then deduplicate the syntax-valid clauses by their normalized ASTs. Finally, Houdini iteratively removes every clause whose establishment or preservation obligation fails under the verifier (Flanagan & Leino, 2001). We denote the surviving set by $\mathsf{Filter}_{\mathcal{L}}(X)$. Filtering can retain compatible clauses from imperfect responses but cannot invent a relation absent from the combined pool.

During RL scoring, sampled reachable states provide a cheap pre-filter for out-of-scope or obviously false clauses, after which the verifier remains the source of soundness. Inference omits this optimization; all uses otherwise share clause splitting, capping, normalization, combination, syntax filtering, and Houdini.

4.3 Compressing Multi-Response Search into SFT Targets

For every masked loop $\mathcal{L}$, we sample a response group, combine its clauses, and compute $\mathsf{Filter}_{\mathcal{L}}(X)$. A nonempty result is serialized as one canonical response:

$$\mathcal{D}_{\mathrm{SFT}} = \left\{ \left(\mathcal{L}, \mathrm{serialize}(\mathsf{Filter}_{\mathcal{L}}(X)) \right) \right\}.$$

This distills a filtered multi-response result into one answer. Neither side contains Q: the input is masked and the output contains only inductive clauses. Execution states are used only for RL, never as SFT answers.

4.4 Execution-Based Training Examples

To rank inductive sets without revealing Q, we derive a strength signal from executions. The sampler instruments one braced `while` loop, compiles it with `gcc`, and records loop-head valuations for a deterministic, precondition-checked input sweep. The recorded states form $\mathcal{P} \subseteq \mathrm{Reach}(\mathcal{L})$; clauses falsified by these states are removed before scoring. If any run hits the iteration cap, the sample is marked incomplete and cannot support terminal or range examples.

The sampler then constructs three kinds of negative examples without reading an assertion or postcondition:

- **Relational perturbations** change one or two variables by $\pm\{1, 2\}$ in densely observed states, testing relations such as $x + y = n$.
- **Post-exit continuations** execute the real loop body for additional steps after a genuine exit.
- **Range perturbations** move one variable outside its observed range and test useful bounds.

Figure 2 illustrates how the three families are generated from observed executions and then converted into negative traces.

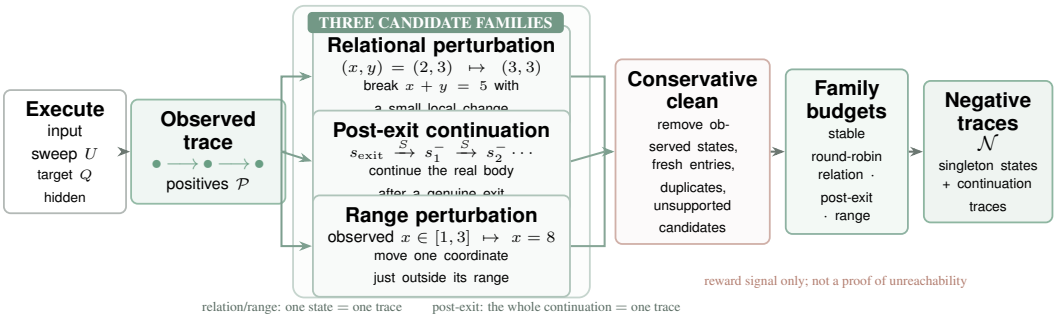

Figure 2: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Three transformations probe relational structure, behavior past a genuine exit, and sampled bounds. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q.

These are training signals, not proofs of global unreachability. We remove observed reachable states, possible fresh loop entries, duplicates, and candidates that depend on unsupported state. Relation and range candidates form singleton negative traces, whereas all states from one post-exit continuation form one trace. Each family has an independent budget, filled round-robin across variable and direction buckets. Pointers, arrays, calls, unsupported locals, or oracle-dependent behavior disable the affected construction; a program with no safe variable contributes positives only. Capped executions contribute positives but disable constructions that require complete terminal or range information. Algorithm 2 in Appendix A.1 and Table 7 give the sampling procedure and exact configuration.

Let $\mathcal{N}$ denote the retained negative traces. One trace can contain several witness states from a perturbed execution. For a candidate set X, let $\mathrm{rej}(X) \subseteq \mathcal{N}$ be the traces on which at least one clause in $\mathsf{Filter}_{\mathcal{L}}(X)$ is false.

4.5 Rewarding Strength and Complementarity

The deployed union needs strong individual responses that remain complementary within a group. We reward both properties.

Soundness is checkable directly: Houdini either establishes and preserves a clause or it does not. Strength is not. An exact invariant need not have a finite representation in our annotation fragment (Section 2), so we cannot target exactness itself, and a binary sound/unsound label cannot tell a response that is merely inductive from one that is inductive and useful: $true$ passes the same check as a tight bound. We therefore need a graded, target-independent proxy for strength that a partial, sound response can satisfy by degree rather than all at once.

For response A_i, let $R_i = \mathrm{rej}(\bar{A}_i)$ after standalone filtering. Its negative rejection rate is

$$\mathrm{coverage}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

This gives dense credit to a useful partial answer without querying the hidden target.

Coverage alone can cause every response to repeat the easiest high-reward clauses. For a group of G responses, define

$$v(S) = \frac{\left|\bigcup_{j\in S} R_j\right|}{|\mathcal{N}|}, \qquad f(n) = \sum_{j=1}^{G} \mathbf{1}[n \in R_j].$$

Here $v(S)$ is the negative-trace coverage of a response coalition. Its closed-form Shapley allocation gives response A_i the group credit

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{n\in R_i} \frac{1}{f(n)}. \tag{1}$$

For a trace rejected by $f(n)$ responses, each receives $1/f(n)$ credit: a unique rejection receives full credit, while redundant rejections share it. Equivalently, $1/f(n)$ is the probability that A_i is the first response to cover n under a uniformly random response ordering. The allocation therefore conserves the combined standalone coverage,

$$\sum_{i=1}^{G} \phi_i = \frac{\left|\bigcup_{i=1}^{G} R_i\right|}{|\mathcal{N}|}.$$

Unlike enumerating general response subsets, Equation 1 uses rejection frequencies already computed for coverage and adds no verifier calls. Individual coverage retains absolute response strength, while ϕ_i assigns the group's combined behavioral coverage without double counting.

We penalize only conservative duplicates within a response,

$$d_{\mathrm{dup}}(A) = |\bar{A}| - |\mathrm{dedup}(\bar{A})|.$$

The key normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations requiring stronger arithmetic assumptions. Unique zero-coverage clauses are not penalized because they may support another clause or the hidden target.

The generation reward is

$$r_{\mathrm{gen}}(A_i) = w_c\,\mathrm{coverage}(A_i) + w_g\,\phi_i - w_d\,d_{\mathrm{dup}}(A_i) - w_o\,o(A_i), \tag{2}$$

with $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$. Clauses beyond K receive no coverage and incur overflow cost. Houdini removes failures before scoring, so one bad clause does not erase useful clauses in the response.

We also compute the rejection rate of $\mathsf{Filter}_{\mathcal{L}}(X_0)$ for the combined pool. It controls group resampling but is not copied to every response. With no safe negatives, reward falls back to binary inductiveness for a nonempty response.

Group-relative policy update. Each masked loop $\mathcal{L}$ samples a group of G token sequences $o_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$ from the current policy; parsing, splitting, and capping (§4.2) turn o_i into the scored response A_i. We normalize r_{gen} within its own sampling group to obtain a group-relative advantage,

$$\hat{A}_i = \frac{r_{\text{gen}}(A_i) - \text{mean}_{1 \le j \le G}\, r_{\text{gen}}(A_j)}{\text{std}_{1 \le j \le G}\, r_{\text{gen}}(A_j) + 10^{-6}},$$

shared by every token of o_i, and update θ with the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(\rho_{i,t}(\theta)\,\hat{A}_i,\ \text{clip}(\rho_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_i\Big) - \beta\, D_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\right], \tag{3}$$

where $\rho_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid \mathcal{L}, o_{i,<t}) \big/ \pi_{\theta_{\text{old}}}(o_{i,t} \mid \mathcal{L}, o_{i,<t})$ and π_{ref} is the frozen reference policy.

Group-relative normalization is why the Shapley term matters for training, not only for evaluation. Coverage alone is bounded, so once sampling collapses onto one easiest clause every response in the group reports nearly the same reward, the group standard deviation shrinks, and $\hat{A}_i$ becomes an uninformative signal. Shapley allocation splits credit for shared rejections and gives full credit to a rejection supplied by only one response. It therefore breaks ties before normalization and rewards a rollout for its allocated contribution to the group's combined coverage. This is still a target-independent behavioral surrogate rather than the Shapley value of final Q-sufficient verification.

4.6 Deployment and the Fixed-Budget Objective

At deployment, the model samples k masked responses, splits and normalizes their clauses, combines them, and runs the syntax and Houdini filters. No execution examples are constructed. Only then are the retained clauses and Q inserted into the untouched program for the final verifier proof.

The retained clause set and the invariant predicate produced by one inference invocation are therefore

$$X_k = \bigcup_{i=1}^{k} \bar{A}_i, \qquad C_k = \mathsf{Filter}_{\mathcal{L}}(X_k), \qquad I_k = \bigwedge_{a \in C_k} a, \qquad A_i \sim \pi(\cdot \mid \mathcal{L}),$$

rather than the best standalone response. SFT compresses a large candidate pool into stronger responses; RL improves their coverage and complementarity. Combine@k and k_{95} measure how few trained-policy responses are needed to match a much larger base-model sample.

5 Experiments

We organize the evaluation around five questions. RQ1 probes how direct pass@k and combined inference scale with the sampling budget. RQ2 compares pass@1 and combine@1 across model families. RQ3 compares the resulting system with specialized invariant-generation tools. RQ4 isolates the improvement from RL under both zero-shot and SFT initialization. RQ5 ablates the reward terms and negative-example sampler.

5.1 Experimental Setup

Benchmarks. We evaluate 832 integer C programs. The *Core-366* subset contains 316 linear programs from Code2Inv, SyGuS, and SV-COMP and 50 nonlinear programs from LIPuS. The remaining 466 programs are the integer fragment of Loopy's original 469-program corpus; three floating-point programs are excluded because they fall outside our scalar-integer predicate language. Unsupported inputs remain failures in the common denominator.

Target-hidden evaluation. Before any model-facing step, we remove `assert` and `ensures` predicates, executable assertion calls, and conventional error-label names. Preconditions and executable loop semantics remain visible. After a method fixes its invariants, they are inserted into the untouched original program. A task succeeds only if the source parses, Frama-C/WP proves establishment and preservation for every retained invariant, and every restored target goal succeeds. Failures, timeouts, malformed outputs, and unsupported programs count as zero.

Verifier and inference. All final checks use Frama-C 31.0 (Gallium) (Kirchner et al., 2015), WP, and Z3 4.13.3. Neural experiments use the same target-hiding code, generation template, clause splitter, semantic deduplication, and Houdini implementation. We use in-process vLLM and each checkpoint's chat template. Open-weight checkpoints are decoded with `enable_thinking=false`; OpenAI-compatible GPT-5 requests use `reasoning_effort=none`. Runs for which the serving interface cannot explicitly disable reasoning are not entered in the main table. Unless an RQ varies k, generation uses temperature 1.0, top-p 1.0, at most 2,048 new tokens, and no re-roll or target-conditioned feedback. We do not set an additional top-k cutoff or repetition penalty beyond the vLLM defaults. For the k-sweep, one batched request draws 100 responses per program, and every pass@k and combine@k point reuses that same saved response pool. We do not fix a per-response sampling seed; the 100 outputs are stochastic draws from that single batch.

Metrics. Pass@k is the probability that at least one of k independent rollouts generates a Q-sufficient invariant set, averaged over programs. We report the standard unbiased pass@k estimate computed from the 100 saved rollouts. For a program with c independently Q-sufficient responses among n saved responses, the estimator is

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}.$$

Merely inductive but Q-insufficient (*sound-but-weak*) sets do not count as successes. Combine@k is the fraction of programs for which unioning the invariants from the first k rollouts and applying Houdini yields a Q-sufficient inductive set. We do not form an unbiased combination estimator by averaging over all k-subsets of the 100 saved rollouts: the $\binom{100}{k}$ subset space is prohibitively large, and every subset would require a separate Houdini and verification run. We therefore evaluate the fixed saved prefix of length k. Pass@k asks whether at least one response is independently Q-sufficient, whereas combine@k asks whether the accumulated clauses jointly contain a Q-sufficient inductive subset. We summarize the sampling budget needed to approach the measured ceiling by

$$k_{95} = \min\{k : C_\pi(k) \geq 0.95\, C_\pi(100)\},$$

where $C_\pi(k)$ is combine@k for policy π.

Pending-result convention. Unfinished result cells below are marked *TBD* and remain empty until the corresponding common-protocol run is complete; populated cells come only from completed runs, and we do not extrapolate missing values from pilots.

5.2 RQ1: How Do pass@k and combine@k Scale?

We sample 100 target-hidden responses per task from five base checkpoints: Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B, and Qwen3-32B (Yang et al., 2025). All points for one checkpoint are computed from the same saved response pool. We evaluate every prefix and report representative budgets together with k_{95}.

Base checkpoint	pass@1	pass@50	pass@100	combine@1	combine@50	combine@100	k_{95}
Qwen3-4B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-30B-A3B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-32B	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 1: Five-base-model probe of direct pass@k and deployed combine@k. The final artifact will additionally release the complete curves for $1 \leq k \leq 100$.

This probe tests two complementary claims. First, combine@k can break the ceiling of direct pass@k: responses that fail independently may still provide different clauses of one proof. Second, combination cannot invent a relation that the model never emits. Small models can therefore remain difficult to use even with aggregation: poor invariant quality lowers the curve's asymptote, while

limited exploratory diversity makes it saturate early. We use the complete curve and k_{95} to test the observed stabilization near $k = 50$, rather than treating wider sampling as an unbounded source of improvement.

5.3 RQ2: How Strong Is combine@1 Across Models?

The main neural experiment evaluates exactly one response per program. Pass@1 treats that response as one all-or-nothing invariant set. Combine@1 splits conjunctions into clauses, removes duplicates, and runs Houdini on the resulting pool. No pass@$k > 1$ or combine@$k > 1$ result is included in this RQ, so model quality is not confounded with a larger inference budget.

Model	pass@1	combine@1	Δ	Time / task	Avg. tokens
`gpt-5-nano`	TBD	TBD	TBD	TBD	TBD
`gpt-5-mini`	TBD	TBD	TBD	TBD	TBD
`gpt-5`	251 (30.17%)	477 (57.33%)	+226 (+27.16 pp)	62.05 s	1,028.12
Gemini-3.5-Flash	TBD	TBD	TBD	TBD	TBD
Claude Sonnet-4.6	TBD	TBD	TBD	TBD	TBD
DeepSeek-V4-Flash	TBD	TBD	TBD	TBD	TBD
Qwen3-14B	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	TBD	TBD	TBD	TBD	TBD
Llama (checkpoint TBD)	TBD	TBD	TBD	TBD	TBD

Table 2: Main cross-model experiment. RQ2 records only pass@1 and combine@1; Δ gives the gain in verified programs and percentage points. Time is the mean end-to-end combine@1 pipeline time, and tokens are the exact mean API usage of the single response shared by pass@1 and combine@1.

Combine@1 isolates a useful property of the framework: even with one model call, a response need not be accepted or rejected as a monolithic object. Houdini can retain its sound clauses while removing a failing clause. The comparison tests whether this lightweight inference step gives particularly large gains to open models, whose raw responses may be less reliable.

5.4 RQ3: How Does LoopGym Compare with Specialized Tools?

We compare against the native target-hidden pipelines of AutoSpec, SESpec, and Clause2Inv, and against Daikon 5.8.24 (Wen et al., 2024; Yang et al., 2026; Cao et al., 2025; Ernst et al., 2007). Clause2Inv supports only the 366 programs with transition verification conditions; all other inputs remain failures in the 832-program denominator. The LoopGym row fixes its model and combine@1 configuration before the comparison rather than selecting the best result per benchmark.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Model calls	Time / task
AutoSpec	TBD	TBD	TBD	TBD	TBD	TBD
SESpec	TBD	TBD	TBD	TBD	TBD	TBD
Clause2Inv	TBD	TBD	TBD	TBD	TBD	TBD
Daikon	TBD	TBD	TBD	TBD	TBD	TBD
LoopGym (`gpt-5-nano`, combine@1)	TBD	TBD	TBD	TBD	TBD	TBD

Table 3: Common target-hidden comparison with specialized tools. Every row uses the same untouched source and Frama-C/WP final judge.

Efficiency and orchestration. LoopGym does not maintain a multi-turn repair conversation, workflow state, or growing context history. Its orchestration state is only a set of independent clauses: generate, split, normalize, union, and filter. We report model calls, generated tokens, verifier invocations, wall time, and peak prompt length so that effectiveness is not separated from workflow cost. This RQ tests whether the simple framework is competitive with specialized systems without their context management, and whether its gains remain substantial on open models.

5.5 RQ4: How Much Does Reinforcement Learning Improve Inference?

We use Qwen3-8B and separate two paired training paths. The zero-shot path compares the base checkpoint with RL-Zero. The supervised path compares the SFT checkpoint immediately before RL with the checkpoint obtained by continuing from it under the same RL objective. Each after-RL model therefore has a matched before-RL initialization, preventing SFT gains from being misattributed to RL.

Initialization	Training stage	pass@1	combine@1	combine@10	combine@100	k_{95}	Verified / 832
Base 8B	before RL (Zero)	TBD	TBD	TBD	TBD	TBD	TBD
Base 8B	after RL (RL-Zero)	TBD	TBD	TBD	TBD	TBD	TBD
SFT 8B	before RL (SFT)	TBD	TBD	TBD	TBD	TBD	TBD
SFT 8B	after RL (SFT+RL)	TBD	TBD	TBD	TBD	TBD	TBD

Table 4: Paired evaluation of RL from zero-shot and SFT initialization. All four rows use identical rollout budgets and the same inference code.

RL is useful only if the after-RL checkpoint improves over its matched before-RL checkpoint. We report pass@1 as a diagnostic, but the primary outcomes are combine@k and k_{95}: the objective is designed to make a small group contain stronger and more complementary clauses, not merely to increase standalone proofs.

5.6 RQ5: What Drives the Training Gain?

Both ablation studies use Qwen3-8B and hold prompts, data order, rollout-group size, update budget, seeds, and inference code fixed.

Reward-function ablation. The binary verified baseline assigns 1 only when a response passes the target-independent inductiveness check and 0 otherwise. *Base* uses negative rejection coverage; *Shapley* allocates the union of standalone negative coverage among the responses that provide it; and *redundancy* adds the conservative semantic-duplicate cost. The cumulative rows measure one additional term at a time.

8B training reward	pass@1	combine@1	combine@5	combine@10	k_{95}	Verified / 832
Binary verified $\{0, 1\}$	TBD	TBD	TBD	TBD	TBD	TBD
Base	TBD	TBD	TBD	TBD	TBD	TBD
Base + Shapley	TBD	TBD	TBD	TBD	TBD	TBD
Base + Shapley + redundancy	TBD	TBD	TBD	TBD	TBD	TBD

Table 5: Cumulative reward ablation on Qwen3-8B.

Negative-sampling ablation. We hold the number of executions and total negative-example budget fixed. *Relation* uses perturbations of witnessed relations. The next row adds post-exit continuations (*escape*); the full sampler additionally adds bound/range violations. A budget-matched random perturbation baseline tests whether structured negatives matter beyond merely increasing sample count.

Negative sampler	Reward variance	Hard-collision rate	pass@1	combine@1	combine@10	Verified / 832
Random (budget matched)	TBD	TBD	TBD	TBD	TBD	TBD
Relation	TBD	TBD	TBD	TBD	TBD	TBD
Relation + escape	TBD	TBD	TBD	TBD	TBD	TBD
Relation + escape + bound/range	TBD	TBD	TBD	TBD	TBD	TBD

Table 6: Negative-example sampling ablation under a fixed sample budget.

Besides final verification, we report reward variance and hard-collision audits. Reward variance tests whether a sampler supplies enough discrimination for group-relative RL. The collision audit checks whether a generated negative is also observed as reachable under expanded execution sampling, guarding against gains obtained from systematically incorrect negative labels.

6 Conclusion and Limitations

Because clauses compose across responses, our objective is the filtered combination, not pass@1. Inference uses rollout–combine–filter before restoring the target. SFT distills a verified combination into one response; target-independent RL rewards response strength and behavior not covered by peers through a Shapley allocation of group coverage. Consequently, ten trained-policy responses outperform one hundred base-model responses even as pass@1 decreases.

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require substantially richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. In particular, our state perturbations cannot yet construct reliably informative negative examples for inductive predicates or interacting nested loops. Extending both the data and negative-example semantics to these settings remains future work (Liu et al., 2024).

Inference engine. In preliminary comparisons, we observed lower verification rates when serving the evaluated checkpoints with vLLM than with alternative inference backends, even under nominally matched decoding settings. We nevertheless use vLLM for all reported experiments: its throughput makes the sampling study practical, and using the same engine for training rollouts and evaluation avoids an additional train–test mismatch. The reported absolute success rates may therefore be conservative and should not be read as an engine-independent ceiling on the checkpoints.

Verifier and specification language. All filtering and final judgments use Frama-C/WP, and candidates are expressed in ACSL. Parser behavior, generated proof obligations, prover integration, and annotation syntax all affect which clauses survive the pipeline. The rollout–combine–filter principle is not intrinsically tied to this toolchain, but the quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

Feedback regime. We focus on the feedback-scarce setting and do not evaluate iterative optimization from verifier feedback. In our pilot runs, a model usually reached *sound but weak* invariants quickly: they passed inductiveness checks, but the verifier provided no signal about the missing behavioral strength while the target remained hidden. Consequently, feedback-based refinement produced little marginal gain, and that gain decreased further as the number of initial rollouts grew because their union already captured most easy corrections. Our results therefore characterize target-independent training and finite-budget combination, not settings where a verifier exposes rich counterexamples or target-directed repair signals.

References

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.

Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. `https://frama-c.com/html/acsl.html`, 2021.

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 84–96, 1978.

Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 443–456, 2013.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.

Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.

Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 281–292, 2008.

Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 634–640. Springer, 2009.

Matthias Heizmann, Jochen Hoenicke, and Andreas Podelski. Software model checking for people who love automata. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pp. 36–52. Springer, 2013. doi: 10.1007/978-3-642-39799-8_2.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 318–329, 2004.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026.

Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A Implementation Details

Table 7 lists the implemented defaults. Execution sampling is used only to compute reinforcement-learning rewards; it does not construct the synthetic SFT corpus. SFT synthesis, RL, and inference share the rollout–combine–filter implementation: the same conjunction splitting, response cap, normalization, candidate combination, iterative syntax filter, optional AST deduplication, and Houdini pruning. SFT serializes the filtered combination, RL scores the responses that populate it, and inference restores Q only after filtering. Inference does not construct execution examples. The operators are shared, but RL applies Houdini once per response for credit assignment whereas SFT and inference apply it after union; Appendix F analyzes this ordering mismatch.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
range-perturbation steps	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	320 (≤ 64 fallback) / 64 / 128	relational / post-exit / range examples; stable round-robin within each kind
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	$(1.0, 0.3)$	negative rejection / Shapley group-credit weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generated response
w_o	0.05	overflow cost per clause after K
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget

Table 7: Defaults for execution sampling (§4.4), reward computation (§4.5), inference, and verification.

The exact near-input tier order is `0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9`.

A.1 Negative-Trace Sampling

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\mathrm{Perturb}_{1,2}$ applies the configured single-variable and pairwise changes, while $\mathrm{RangeEscape}$ moves one variable just outside its sampled range. The exact perturbations and budgets are those in Table 7.

Input: masked loop $\mathcal{L} = (P, B, S)$, input schedule U, configuration Θ
Output: reachable states $\mathcal{P}$, negative traces $\mathcal{N}$

$\mathcal{T}, \mathcal{C}_{\text{post}}, c \leftarrow \text{Trace}(\mathcal{L}, U, \Theta)$ ▷ *c: some run was capped*
$\mathcal{P} \leftarrow \text{UniqueHeads}(\mathcal{T})$; $M \leftarrow \text{SafeModifiedVars}(S)$
if $M = \varnothing$ or S has unsupported state **then return** $(\mathcal{P}, \varnothing)$
$\mathcal{S} \leftarrow \text{StratifiedSample}(\mathcal{P})$
$C_{\text{rel}} \leftarrow \text{Perturb}_{1,2}(\text{Dense}(\mathcal{S}), M, \Theta)$
if $\neg c$ **then** $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_{1,2}(\text{Terminals}(\mathcal{T}), M, \Theta)$
$C_{\text{range}} \leftarrow \text{RangeEscape}(\mathcal{S}, M, \mathcal{P}, \Theta)$ if $\neg c$, else $\varnothing$
$(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}) \leftarrow \text{Clean}_{\mathcal{P}}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}})$
$N_{\text{rel}}, \mathcal{N}_{\text{post}}, N_{\text{range}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}, \Theta)$
$\mathcal{N} \leftarrow \{\langle s \rangle : s \in N_{\text{rel}}\} \cup \mathcal{N}_{\text{post}} \cup \{\langle s \rangle : s \in N_{\text{range}}\}$
return $(\mathcal{P}, \mathcal{N})$

Algorithm 2: Target-hidden negative-trace sampling

$\text{Clean}_{\mathcal{P}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates, using family priority relation, post-exit, then range. A cleaned post-exit continuation remains one trace; relation and range candidates become singleton traces. Within the relation budget, candidates that preserve the guard truth value and remain inside the sampled coordinate ranges are selected first.

B Generation Prompt

The same generation template proposes the responses later combined during SFT synthesis, RL, and inference. Any assertion is stripped before the program is inserted. General annotation rules are supplied separately by `system_prompt.txt`.

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

C Training Interface

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and Z3. It exposes `POST /reward`, accepting program source and an array of model responses. The response exposes the Shapley credit explicitly:

```
{program, n_positives, n_negatives, filter_mode,
 reward_mode, rollout_rewards[], base[], shapley_credit[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score, should_reroll}
```

Responses may be raw LLM text, invariant lists, or annotated code; parsing is uniform. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

Table 8: Complete probe results. k_{95} is the smallest k at which combine@k reaches 95% of its value at $k = 100$.

Model	Metric	$k=1$	$k=5$	$k=10$	$k=20$	$k=30$	$k=50$	$k=100$	k_{95}
Qwen3-4B	pass	TBD	TBD	TBD	TBD	TBD	TBD	TBD	–
	combine	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	pass	TBD	TBD	TBD	TBD	TBD	TBD	TBD	–
	combine	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B	pass	TBD	TBD	TBD	TBD	TBD	TBD	TBD	–
	combine	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-30B-A3B	pass	TBD	TBD	TBD	TBD	TBD	TBD	TBD	–
	combine	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-32B	pass	TBD	TBD	TBD	TBD	TBD	TBD	TBD	–
	combine	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 9: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Path	Model	pass@1	combine@1	combine@5	combine@10	combine@50	combine@100	k_{95}
Zero	Base 8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Zero	Base 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT	SFT 8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT	SFT 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD

D Complete Experimental Data

This section reports the complete experimental-data schema used for the final paper. We include per-model, per-suite, and per-seed fields so that aggregate numbers in the main text can be traced to their underlying measurements. Entries marked TBD will be filled from the final evaluation artifacts; no missing value is used in the analysis.

D.1 Probe Curves

D.2 Reinforcement-Learning Results

D.3 Ablation Results

D.4 Cross-Model Main Results

D.5 Comparison with Specialized Tools

Table 10: Complete reward-ablation results on the 8B model. Reported values are verification rates; Verified $\in \{0, 1\}$ records whether the final union is accepted.

Reward	Seed 1	Seed 2	Seed 3	Mean	Std.	Verified
Binary verification	TBD	TBD	TBD	TBD	TBD	TBD
Base	TBD	TBD	TBD	TBD	TBD	TBD
Base + Shapley	TBD	TBD	TBD	TBD	TBD	TBD
Base + Shapley + redundancy	TBD	TBD	TBD	TBD	TBD	TBD

Table 11: Complete negative-trace sampler ablation on the 8B model.

Sampler	Seed 1	Seed 2	Seed 3	Mean	Std.	Verified
Random	TBD	TBD	TBD	TBD	TBD	TBD
Relation	TBD	TBD	TBD	TBD	TBD	TBD
Relation + escape	TBD	TBD	TBD	TBD	TBD	TBD
Relation + escape + bound/range	TBD	TBD	TBD	TBD	TBD	TBD

Table 12: Complete main results. Each benchmark column reports pass@1/combine@1.

Model	Code2Inv	SyGuS	SV-COMP	LIPuS	Loopy	Overall	Time	Output tokens
GPT-5-nano	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GPT-5-mini	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GPT-5	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemini-3.5-Flash	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Claude-Sonnet-4.6	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
DeepSeek-V4-Flash	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama (checkpoint TBD)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 13: Complete tool-comparison results. Verification rate is reported for each benchmark and overall; costs are measured over the same benchmark split.

Method	Code2Inv	SyGuS	SV-COMP	LIPuS	Loopy	Overall	Model calls	Verifier calls	Time	Peak context
AutoSpec	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SESpec	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Clause2Inv	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Daikon	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
LoopGym (GPT-5-nano, combine@1)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

E WHY THINKING IS DISABLED

We disable chain-of-thought (CoT) in data synthesis, training, and inference. Open-weight checkpoints use `enable_thinking=false`, and compatible hosted models use `reasoning_effort=none`. A run is excluded from the main comparison if its serving interface cannot explicitly disable reasoning. This choice has two practical motivations.

CoT substantially increases generation cost. Our inference procedure samples many rollouts for each program. Explicit CoT adds reasoning tokens and generation latency to every rollout, so its cost is multiplied by the sampling budget. The required output, however, is only a short, machine-parseable set of ACSL clauses. We therefore omit explicit CoT to keep both data generation and multi-rollout inference efficient.

Invariant clauses compose, but CoTs do not. A single rollout may contain both valid and invalid candidate invariants. The invariant output is explicitly decomposable into clauses: Houdini can remove a failing clause while retaining the useful clauses, and the retained clauses can then be combined across rollouts. A CoT does not have this property. Correct and incorrect reasoning may be interleaved in one trace, with no reliable clause-level correspondence that would let us remove the reasoning associated with a rejected clause. Nor is there a principled way to merge the remaining fragments from different rollouts into one faithful CoT for the combined invariant set. Generating a new rationale after filtering would only produce a post-hoc explanation. Consequently, CoT supervision is not aligned with the clause-wise filter-and-combine target. For consistency, CoT remains disabled during synthetic-data construction, RL training, and inference.

F Training–Inference Mismatch in Houdini Ordering

Let A_i be the invariant set proposed by rollout i, and let $\mathsf{H}(X)$ denote syntax filtering followed by Houdini on candidate set X. SFT construction and inference first merge the candidates and then filter them:

$$S_{\text{infer}} = \mathsf{H}\left(\bigcup_{i=1}^{G} A_i\right). \tag{4}$$

This order allows a clause from one response to be preserved with the help of a companion clause proposed by another response.

RL requires a scalar reward for each rollout. The current implementation therefore runs Houdini independently and computes coverage, Shapley credit, and redundancy from

$$S_i^{\text{train}} = \mathsf{H}(A_i). \tag{5}$$

It also evaluates $\mathsf{H}(\bigcup_i A_i)$ as a group-level diagnostic and for reroll decisions, but this shared result does not directly determine each rollout's token-level advantage.

Why the two orders are not equivalent. Houdini is not distributive over union:

$$\mathsf{H}\left(\bigcup_i A_i\right) \neq \bigcup_i \mathsf{H}(A_i) \quad \text{in general.} \tag{6}$$

For example, a clause from one rollout may fail preservation in isolation but become inductive when conjoined with a supporting clause from another rollout. Thus, two clauses rejected by separate Houdini runs can belong to an inductive conjunction after union. Because the training reward filters each response first, it misses the positive credit associated with this cross-rollout support.

Why the practical effect is small. This is missed positive credit, not an explicit penalty for non-inductiveness. The reward assigns coverage and Shapley credit to surviving clauses and penalizes duplicates and overflow, but it does not penalize a unique clause merely because it fails the standalone Houdini check. Such a clause therefore receives no inductiveness-derived reward, yet the policy is not directly trained to suppress it. At inference time, raw candidates from all responses are unioned before Houdini, so a complementary pair remains available and can be retained jointly. The mismatch therefore does not change the inference procedure or remove cross-response support from its candidate pool. In our experiments, cases rescued only by cross-rollout inductiveness occur with low probability, making this a small mismatch in practice.

Why retain this approximation. Independent filtering gives a stable, parallelizable reward for every response. Assigning the same union-level score to all G responses is uninformative under group-relative normalization because the shared component cancels. Exact leave-one-out credit,

$$\Delta_i = V\left(\mathsf{H}\left(\bigcup_j A_j\right)\right) - V\left(\mathsf{H}\left(\bigcup_{j\neq i} A_j\right)\right),$$

requires at least $G+1$ union-level Houdini evaluations per prompt and can still miss higher-order interactions. Shapley-style credit captures such interactions more faithfully but requires many subset evaluations. This would be the Shapley value of the union-and-Houdini verifier game, unlike the closed-form Shapley allocation of negative-set coverage used by our reward.

Implications and future correction. The ordering difference makes standalone Houdini a slightly imperfect reward surrogate for combine@k, but it does not alter inference-time union-and-Houdini. Future work can report the rescue rate represented by the gap between the two sides of Equation 6 and test leave-one-out or randomized-subset credit when that rate is non-negligible. For the present experiments, the observed rarity of these rescues does not justify the additional verifier cost.